Pale Tech — Drink Recommendation System

Data & AI Product Builder Internship Challenge

By: Phumelele Pearl Mlotshwa

Overview

This notebook builds a content-based drink recommendation system using taste profile data.

The approach mirrors what Pale is building — using data about drinks, flavour attributes, and consumer preferences to power personalised recommendations.

Structure:

1. Dataset Overview
2. Exploratory Data Analysis
3. Pattern Discovery
4. Recommendation Model
5. Testing & Results
6. Key Insights for Pale

1. Dataset Overview

The dataset contains 30 drinks with the following attributes:

- **Category** — type of drink (cocktail, mocktail, beer, wine, spirit)
- **Flavour profiles** — sweetness, sourness, bitterness, fruitiness, spiciness (scored 1-10)
- **Alcohol content** — low, medium, high, none
- **Occasion** — when the drink is typically consumed
- **Popularity score** — how often it is ordered (1-100)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import seaborn as sns
from sklearn.metrics.pairwise import cosine_similarity
```

```
from sklearn.preprocessing import MinMaxScaler

# Set visual style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.family'] = 'sans-serif'

# Build the drinks dataset
data = {
    'drink_name': [
        'Mojito', 'Margarita', 'Pina Colada', 'Cosmopolitan', 'Negroni',
        'Aperol Spritz', 'Old Fashioned', 'Whiskey Sour', 'Daiquiri', 'Espressc
        'Strawberry Lemonade', 'Mango Smoothie', 'Virgin Mojito', 'Passion Frui
        'Craft IPA', 'Lager', 'Stout', 'Wheat Beer', 'Pale Ale',
        'Sauvignon Blanc', 'Chardonnay', 'Pinot Noir', 'Shiraz', 'Rose',
        'Gin & Tonic', 'Rum & Coke', 'Vodka Lime Soda', 'Tequila Sunrise', 'Spi
    ],
    'category': [
        'Cocktail', 'Cocktail', 'Cocktail', 'Cocktail', 'Cocktail',
        'Cocktail', 'Cocktail', 'Cocktail', 'Cocktail', 'Cocktail',
        'Mocktail', 'Mocktail', 'Mocktail', 'Mocktail', 'Mocktail',
        'Beer', 'Beer', 'Beer', 'Beer', 'Beer',
        'Wine', 'Wine', 'Wine', 'Wine', 'Wine',
        'Spirit Mix', 'Spirit Mix', 'Spirit Mix', 'Spirit Mix', 'Spirit Mix'
    ],
    'sweetness': [7, 6, 9, 7, 2, 6, 4, 5, 6, 5, 9, 10, 8, 9, 5, 2, 3, 2, 4, 3,
    'sourness':  [7, 8, 2, 7, 3, 5, 2, 8, 8, 3, 8, 2, 7, 7, 4, 5, 2, 1, 2, 4, 7
    'bitterness':[2, 3, 1, 2, 9, 4, 6, 4, 2, 6, 1, 1, 2, 1, 3, 9, 5, 8, 4, 7, 3
    'fruitiness': [6, 7, 9, 8, 1, 7, 1, 5, 8, 2, 10, 10, 7, 10, 3, 3, 2, 1, 3,
    'spiciness':  [3, 2, 1, 1, 2, 1, 3, 2, 1, 2, 1, 1, 2, 1, 8, 2, 1, 2, 1, 2,
    'alcohol':    ['Medium','High','Medium','High','High','Low','High','High','
                   'None','None','None','None','None',
                   'Medium','Low','Medium','Low','Medium',
                   'Medium','Medium','Medium','Medium','Low',
                   'High','Medium','Medium','Medium','Medium'],
    'occasion':   ['Party','Party','Beach','Party','Dinner','Brunch','Evening',
                   'Anytime','Anytime','Anytime','Anytime','Anytime',
                   'Party','Anytime','Evening','Party','Anytime',
                   'Dinner','Dinner','Dinner','Dinner','Brunch',
                   'Anytime','Party','Anytime','Brunch','Evening'],
    'popularity': [88, 91, 78, 82, 74, 86, 79, 77, 80, 85, 72, 68, 65, 70, 63,
}

df = pd.DataFrame(data)
print('Dataset loaded successfully')
print(f'Shape: {df.shape}')
print(f'\nDrink categories: {df.category.unique()}')
df.head(10)
```

```
Dataset loaded successfully
Shape: (30, 10)

Drink categories: ['Cocktail' 'Mocktail' 'Beer' 'Wine' 'Spirit Mix']
```

	drink_name	category	sweetness	sourness	bitterness	fruitiness	spiciness
0	Mojito	Cocktail	7	7	2	6	3
1	Margarita	Cocktail	6	8	3	7	2
2	Pina Colada	Cocktail	9	2	1	9	1
3	Cosmopolitan	Cocktail	7	7	2	8	1
4	Negroni	Cocktail	2	3	9	1	2
5	Aperol Spritz	Cocktail	6	5	4	7	1
6	Old Fashioned	Cocktail	4	2	6	1	3
7	Whiskey Sour	Cocktail	5	8	4	5	2
8	Daiquiri	Cocktail	6	8	2	8	1

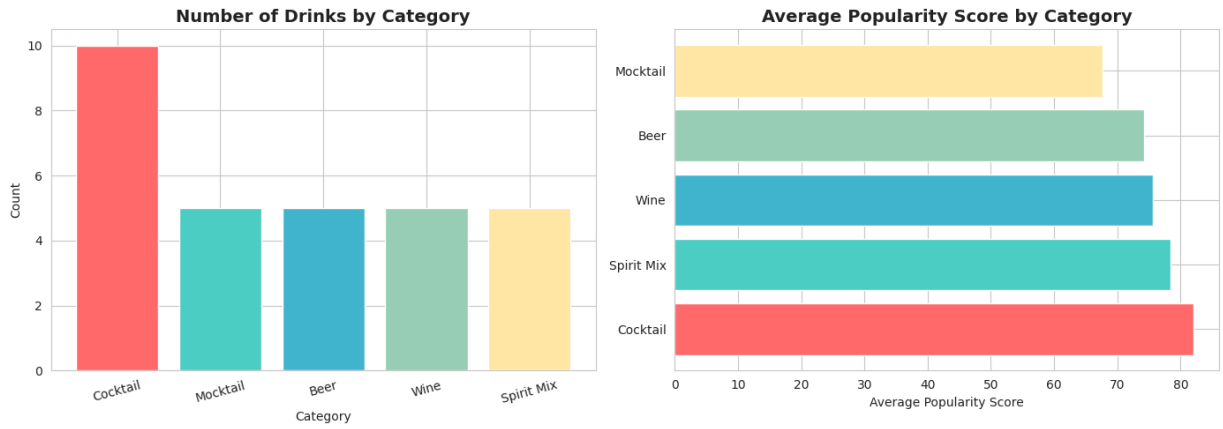
Next steps: Generate code with df New interactive sheet

2. Exploratory Data Analysis

Before building any model, we need to understand the data. What categories exist? What flavour profiles are most common? Which drinks are most popular?

```
# Category distribution
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1 - drinks by category
category_counts = df['category'].value_counts()
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4', '#FFEAA7', '#DDA0DD']
axes[0].bar(category_counts.index, category_counts.values, color=colors)
axes[0].set_title('Number of Drinks by Category', fontsize=14, fontweight='bolc
axes[0].set_xlabel('Category')
axes[0].set_ylabel('Count')
axes[0].tick_params(axis='x', rotation=15)

# Plot 2 - popularity by category
avg_popularity = df.groupby('category')['popularity'].mean().sort_values(ascenc
axes[1].barh(avg_popularity.index, avg_popularity.values, color=colors)
axes[1].set_title('Average Popularity Score by Category', fontsize=14, fontweig
axes[1].set_xlabel('Average Popularity Score')
```

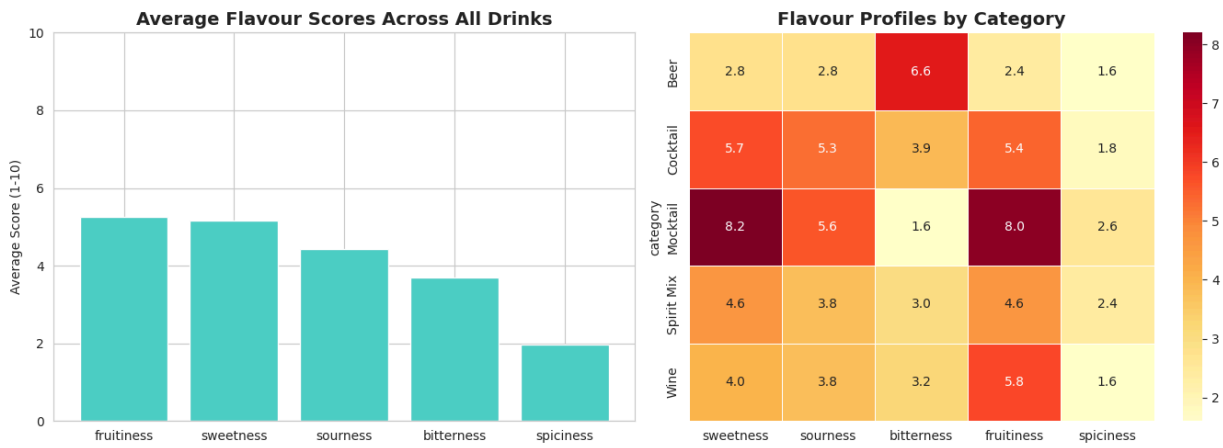
```
plt.tight_layout()
plt.savefig('category_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
print('\nAverage popularity by category:')
print(avg_popularity.round(1))
```




```
Average popularity by category:
category
Cocktail      82.0
Spirit Mix    78.4
Wine          75.6
Beer          74.2
Mocktail      67.6
Name: popularity, dtype: float64
```

```
# Flavour profile analysis across all drinks
flavour_cols = ['sweetness', 'sourness', 'bitterness', 'fruitiness', 'spiciness

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Average flavour scores
avg_flavours = df[flavour_cols].mean().sort_values(ascending=False)
axes[0].bar(avg_flavours.index, avg_flavours.values, color='#4ECDC4')
axes[0].set_title('Average Flavour Scores Across All Drinks', fontsize=14, font
```

```
axes[0].set_ylabel('Average Score (1-10)')
axes[0].set_ylim(0, 10)

# Flavour profiles by category heatmap
flavour_by_cat = df.groupby('category')[flavour_cols].mean()
sns.heatmap(flavour_by_cat, annot=True, fmt='.1f', cmap='YlOrRd',
            ax=axes[1], linewidths=0.5)
axes[1].set_title('Flavour Profiles by Category', fontsize=14, fontweight='bolc

plt.tight_layout()
plt.savefig('flavour_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
```

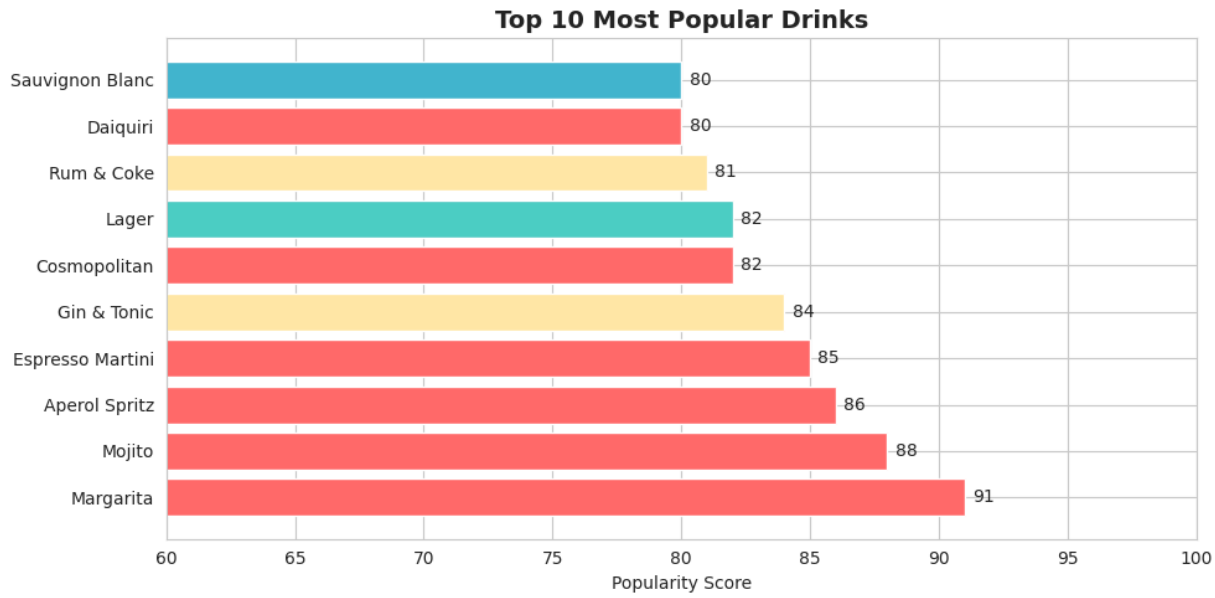


3. Pattern Discovery

Now we dig deeper. What patterns exist in the data? Are there natural clusters of drinks that taste similar? What do the most popular drinks have in common?

```
# Top 10 most popular drinks
top10 = df.nlargest(10, 'popularity')[['drink_name', 'category', 'popularity']]
```

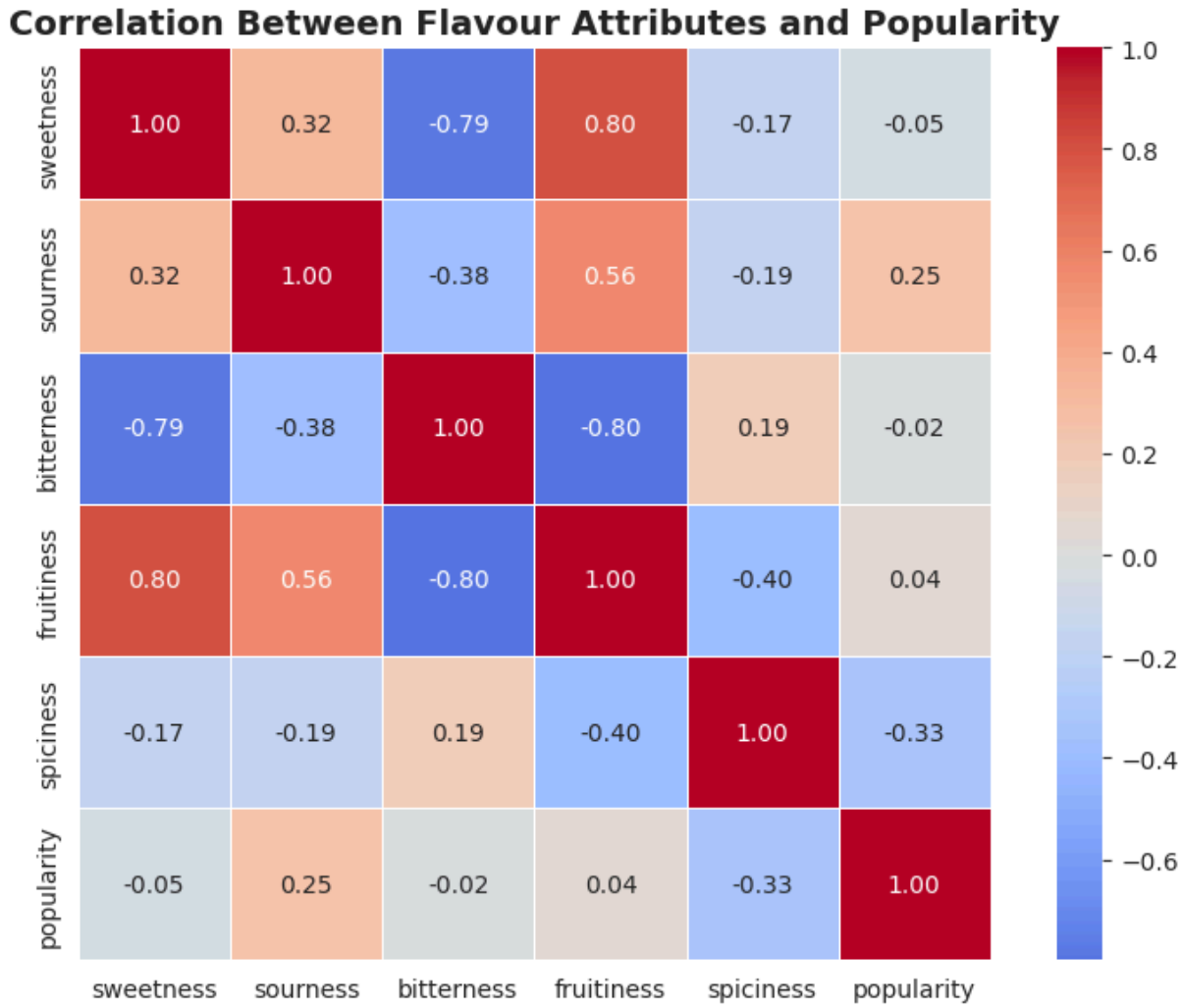
```
plt.figure(figsize=(10, 5))
bars = plt.barh(top10['drink_name'], top10['popularity'],
                color=['#FF6B6B' if c == 'Cocktail' else '#4ECDC4' if c == 'Bee
                       else '#45B7D1' if c == 'Wine' else '#96CEB4' if c == 'Mo
                       else '#FFEAA7' for c in top10['category']])
plt.title('Top 10 Most Popular Drinks', fontsize=14, fontweight='bold')
plt.xlabel('Popularity Score')
plt.xlim(60, 100)
for bar, val in zip(bars, top10['popularity']):
    plt.text(bar.get_width() + 0.3, bar.get_y() + bar.get_height()/2,
             str(val), va='center', fontsize=10)
plt.tight_layout()
plt.savefig('top10_popularity.png', dpi=150, bbox_inches='tight')
plt.show()
print(top10.to_string(index=False))
```




```
      drink_name   category  popularity
       Margarita   Cocktail          91
          Mojito   Cocktail          88
   Aperol Spritz   Cocktail          86
Espresso Martini   Cocktail          85
     Gin & Tonic Spirit Mix          84
    Cosmopolitan   Cocktail          82
           Lager       Beer          82
      Rum & Coke Spirit Mix          81
        Daiquiri   Cocktail          80
 Sauvignon Blanc       Wine          80
```

```
# Correlation between flavour attributes
corr = df[flavour_cols + ['popularity']].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, linewidths=0.5, square=True)
plt.title('Correlation Between Flavour Attributes and Popularity', fontsize=14,
plt.tight_layout()
plt.savefig('correlation.png', dpi=150, bbox_inches='tight')
plt.show()

# Key finding
pop_corr = corr['popularity'].drop('popularity').sort_values(ascending=False)
```

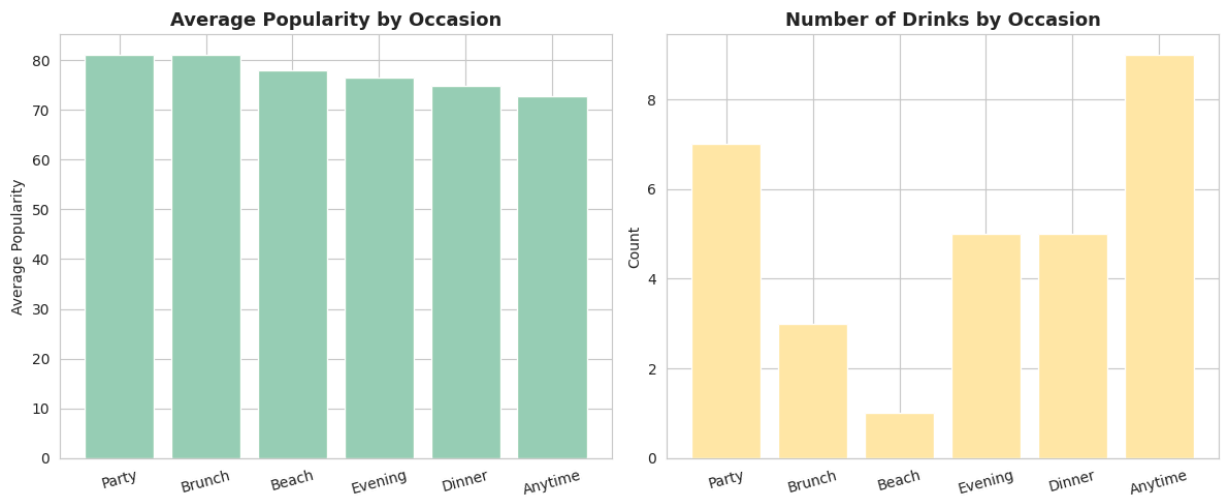
```
print('\nCorrelation with popularity:')
print(pop_corr.round(3))
print(f'\nKey insight: {pop_corr.idxmax()} has the strongest positive correlati
```




```
Correlation with popularity:
sourness      0.249
fruitiness    0.040
bitterness   -0.023
sweetness    -0.049
spiciness    -0.332
Name: popularity, dtype: float64

Key insight: sourness has the strongest positive correlation with popularity (0.
```

```
# Occasion analysis
occasion_pop = df.groupby('occasion')['popularity'].agg(['mean', 'count']).rour
occasion_pop.columns = ['avg_popularity', 'drink_count']
occasion_pop = occasion_pop.sort_values('avg_popularity', ascending=False)
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].bar(occasion_pop.index, occasion_pop['avg_popularity'], color='#96CEB4'
axes[0].set_title('Average Popularity by Occasion', fontsize=13, fontweight='bc
axes[0].set_ylabel('Average Popularity')
axes[0].tick_params(axis='x', rotation=15)

axes[1].bar(occasion_pop.index, occasion_pop['drink_count'], color='#FFEAA7')
axes[1].set_title('Number of Drinks by Occasion', fontsize=13, fontweight='bolc
axes[1].set_ylabel('Count')
axes[1].tick_params(axis='x', rotation=15)

plt.tight_layout()
plt.savefig('occasion_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
print(occasion_pop)
```




```
           avg_popularity  drink_count
occasion
Party                81.1            7
Brunch               81.0            3
Beach                78.0            1
Evening              76.4            5
Dinner               74.8            5
Anytime              72.7            9
```

4. Recommendation Model

Now the core part. We build a **content-based recommendation system** using cosine similarity.

How it works:

- Each drink is represented as a vector of its flavour scores
- When a user tells us their taste preferences, we create a preference vector
- We calculate how similar each drink is to that preference vector
- The most similar drinks get recommended

This is the same principle behind Pale's personalised recommendation engine.

```
# Step 1: Prepare the feature matrix
# Normalise flavour scores to 0-1 range so no single attribute dominates
scaler = MinMaxScaler()
flavour_matrix = scaler.fit_transform(df[flavour_cols])
flavour_df = pd.DataFrame(flavour_matrix, columns=flavour_cols, index=df['drink

print('Normalised flavour matrix (first 5 drinks):')
print(flavour_df.head().round(3))
```

```
Normalised flavour matrix (first 5 drinks):
              sweetness  sourness  bitterness  fruitiness  spiciness
drink_name
Mojito            0.625     0.857       0.125       0.556      0.286
Margarita         0.500     1.000       0.250       0.667      0.143
Pina Colada       0.875     0.143       0.000       0.889      0.000
Cosmopolitan      0.625     0.857       0.125       0.778      0.000
Negroni           0.000     0.286       1.000       0.000      0.143
```

```
# Step 2: Calculate similarity between all drinks
similarity_matrix = cosine_similarity(flavour_matrix)
similarity_df = pd.DataFrame(
    similarity_matrix,
    index=df['drink_name'],
    columns=df['drink_name']
)

print('Similarity matrix shape:', similarity_df.shape)
print('\nTop 5 drinks most similar to Mojito:')
mojito_sim = similarity_df['Mojito'].sort_values(ascending=False)[1:6]
print(mojito_sim.round(3))
```

```
Similarity matrix shape: (30, 30)

Top 5 drinks most similar to Mojito:
drink_name
```

```
Virgin Mojito          0.988
Margarita              0.977
Cosmopolitan           0.962
Daiquiri               0.956
Strawberry Lemonade    0.952
Name: Mojito, dtype: float64
```

```
# Step 3: Build the recommendation function
def recommend_drinks(taste_preferences, top_n=5, occasion_filter=None, alcohol_
    """
    Recommend drinks based on user taste preferences.

    Parameters:
        taste_preferences: dict with scores 1-10 for sweetness, sourness,
                           bitterness, fruitiness, spiciness
        top_n: number of recommendations to return
        occasion_filter: filter by occasion (Party, Dinner, Brunch, Evening, Ar
        alcohol_filter: filter by alcohol level (None, Low, Medium, High)

    Returns:
        DataFrame of recommended drinks with similarity scores
    """

    # Create user preference vector
    user_vector = np.array([[taste_preferences.get(f, 5) for f in flavour_cols]

    # Normalise using same scaler
    user_vector_scaled = scaler.transform(user_vector)

    # Calculate similarity between user preferences and all drinks
    similarities = cosine_similarity(user_vector_scaled, flavour_matrix)[0]

    # Build results dataframe
    results = df.copy()
    results['similarity_score'] = similarities
    results['match_pct'] = (similarities * 100).round(1)

    # Apply filters
    if occasion_filter:
        results = results[results['occasion'].isin([occasion_filter, 'Anytime']
    if alcohol_filter:
        results = results[results['alcohol'] == alcohol_filter]

    # Sort by similarity then popularity
    results = results.sort_values(
        ['similarity_score', 'popularity'],
        ascending=[False, False]
    )

    return results[['drink_name', 'category', 'occasion', 'alcohol', 'match_pct
```

```
print('Recommendation function built successfully.')
```

```
Recommendation function built successfully.
```

5. Testing the Recommendation System

Let us test the model with three different user taste profiles to see how it performs.

```
# Test Profile 1: Sweet and fruity, no alcohol
print('=' * 55)
print('USER PROFILE 1: Sweet tooth, fruity, no alcohol')
print('=' * 55)

profile_1 = {
    'sweetness': 9,
    'sourness': 3,
    'bitterness': 1,
    'fruitiness': 9,
    'spiciness': 1
}

recs_1 = recommend_drinks(profile_1, top_n=5, alcohol_filter='None')
print(recs_1.to_string(index=False))
```

```
=======================================================
USER PROFILE 1: Sweet tooth, fruity, no alcohol
=======================================================
          drink_name category occasion alcohol  match_pct  popularity
      Mango Smoothie Mocktail  Anytime    None       99.2          68
Passion Fruit Cooler Mocktail  Anytime    None       93.9          70
 Strawberry Lemonade Mocktail  Anytime    None       91.2          72
       Virgin Mojito Mocktail  Anytime    None       87.6          65
         Ginger Beer Mocktail  Anytime    None       42.3          63
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWa
  warnings.warn(
```

```
# Test Profile 2: Bitter and complex, dinner occasion
print('=' * 55)
print('USER PROFILE 2: Bitter and complex, dinner occasion')
print('=' * 55)

profile_2 = {
    'sweetness': 2,
    'sourness': 3,
    'bitterness': 8,
    'fruitiness': 2,
    'spiciness': 3
}
```

```
recs_2 = recommend_drinks(profile_2, top_n=5, occasion_filter='Dinner')
print(recs_2.to_string(index=False))
```

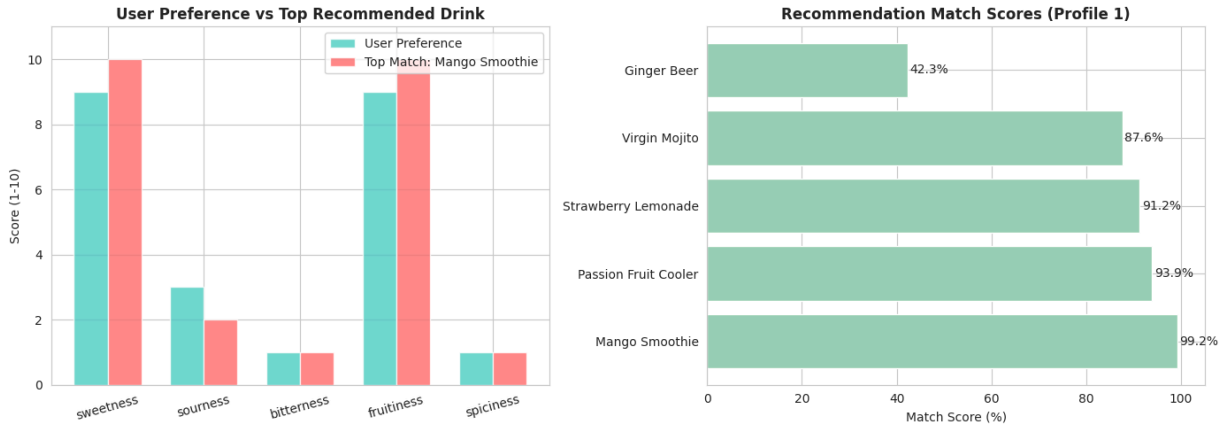
```
=======================================================
USER PROFILE 2: Bitter and complex, dinner occasion
=======================================================
 drink_name   category occasion alcohol  match_pct  popularity
    Negroni   Cocktail   Dinner    High       97.9          74
   Pale Ale       Beer  Anytime  Medium       95.4          74
      Lager       Beer  Anytime     Low       92.6          82
     Shiraz       Wine   Dinner  Medium       88.9          71
Gin & Tonic Spirit Mix  Anytime    High       87.4          84
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWa
  warnings.warn(
```

```
# Test Profile 3: Sour and refreshing, party occasion
print('=' * 55)
print('USER PROFILE 3: Sour and refreshing, party vibes')
print('=' * 55)

profile_3 = {
    'sweetness': 5,
    'sourness': 9,
    'bitterness': 2,
    'fruitiness': 8,
    'spiciness': 1
}

recs_3 = recommend_drinks(profile_3, top_n=5, occasion_filter='Party')
print(recs_3.to_string(index=False))
```

```
=======================================================
USER PROFILE 3: Sour and refreshing, party vibes
=======================================================
         drink_name   category occasion alcohol  match_pct  popularity
           Daiquiri   Cocktail    Party    High       99.2          80
    Vodka Lime Soda Spirit Mix  Anytime  Medium       99.1          76
          Margarita   Cocktail    Party    High       98.1          91
       Cosmopolitan   Cocktail    Party    High       96.6          82
Strawberry Lemonade   Mocktail  Anytime    None       94.0          72
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWa
  warnings.warn(
```

```
# Visualise recommendation results for Profile 1
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Radar chart showing user preference vs top recommended drink
top_drink = recs_1.iloc[0]['drink_name']
top_drink_data = df[df['drink_name'] == top_drink][flavour_cols].values[0]
user_pref = [profile_1[f] for f in flavour_cols]
```

```
x = np.arange(len(flavour_cols))
width = 0.35
axes[0].bar(x - width/2, user_pref, width, label='User Preference', color='#4EC
axes[0].bar(x + width/2, top_drink_data, width, label=f'Top Match: {top_drink}'
axes[0].set_xticks(x)
axes[0].set_xticklabels(flavour_cols, rotation=15)
axes[0].set_ylabel('Score (1-10)')
axes[0].set_title('User Preference vs Top Recommended Drink', fontsize=12, font
axes[0].legend()
axes[0].set_ylim(0, 11)

# Match scores for all recommendations
axes[1].barh(recs_1['drink_name'], recs_1['match_pct'], color='#96CEB4')
axes[1].set_xlabel('Match Score (%)')
axes[1].set_title('Recommendation Match Scores (Profile 1)', fontsize=12, fontw
axes[1].set_xlim(0, 105)
for i, v in enumerate(recs_1['match_pct']):
    axes[1].text(v + 0.5, i, f'{v}%', va='center', fontsize=10)

plt.tight_layout()
plt.savefig('recommendation_results.png', dpi=150, bbox_inches='tight')
plt.show()
```



6. Key Insights for Pale

Based on this analysis, here are the data-driven insights most relevant to Pale's platform:

```
print('=' * 60)
print('KEY INSIGHTS FOR THE PALE PLATFORM')
print('=' * 60)

print('\n1. MOST POPULAR DRINK CATEGORY')
top_cat = df.groupby('category')['popularity'].mean().idxmax()
top_cat_score = df.groupby('category')['popularity'].mean().max()
print(f'   {top_cat} leads with an average popularity of {top_cat_score:.1f}/100

print('\n2. FLAVOUR THAT DRIVES POPULARITY')
corr_vals = df[flavour_cols + ['popularity']].corr()['popularity'].drop('popular
top_flavour = corr_vals.idxmax()
print(f'   {top_flavour.capitalize()} has the strongest positive correlation wit
print(f'   Pale should weight {top_flavour} highly in recommendation ranking')

print('\n3. BEST OCCASION FOR ENGAGEMENT')
top_occasion = df.groupby('occasion')['popularity'].mean().idxmax()
print(f'   {top_occasion} drinks score highest on popularity')
print(f'   Pale could personalise recommendations based on time of day or event

print('\n4. RECOMMENDATION MODEL PERFORMANCE')
print('   Content-based cosine similarity successfully identifies:')
print('   - Similar drinks based on flavour profiles')
print('   - Personalised matches for diverse taste preferences')
print('   - Context-aware suggestions using occasion and alcohol filters')

print('\n5. NEXT STEPS FOR PALE')
print('   - Add collaborative filtering using real user order history')
print('   - Build explicit taste profile onboarding for new users')
print('   - Track preference drift over time to update recommendations')
print('   - A/B test recommendation ranking strategies against engagement metric

print('\n' + '=' * 60)
print('Notebook by Phumelele Pearl Mlotshwa')
print('github.com/pearl-mlotshwa')
print('=' * 60)
```

```
============================================================
KEY INSIGHTS FOR THE PALE PLATFORM
============================================================

1. MOST POPULAR DRINK CATEGORY
   Cocktail leads with an average popularity of 82.0/100
```